📊 Prefix Sum Master Guide

Master the Art of Subarray Calculations with Visual Explanations

📋 Table of Contents

1. What is Prefix Sum?
2. When to Use Prefix Sum
3. Prefix Sum vs Kadane vs Sliding Window
4. The Master Template: Red vs Green
5. Problem 1: Find Pivot Index
6. Problem 2: Subarray Sum Equals K
7. Problem 3: Subarray Sums Divisible by K
8. Problem 4: Contiguous Array
9. Quick Reference Summary

1. What is Prefix Sum? 🤔

Simple Definition:

Prefix Sum stores the **cumulative sum** from the start up to each index. This enables **O(1) subarray sum queries** and works with negative numbers!

Visual Explanation:

```
📊 BUILDING A PREFIX SUM ARRAY

Original Array:  [ 2,  4,  1,  5,  3]
Index:             0   1   2   3   4

Step-by-step building Prefix:

Index 0: prefix[0] = 2
         ●
         2

Index 1: prefix[1] = prefix[0] + arr[1] = 2 + 4 = 6
         ●───●
         2   6

Index 2: prefix[2] = prefix[1] + arr[2] = 6 + 1 = 7
         ●───●───●
         2   6   7

Index 3: prefix[3] = prefix[2] + arr[3] = 7 + 5 = 12
         ●───●───●───●
         2   6   7  12

Index 4: prefix[4] = prefix[3] + arr[4] = 12 + 3 = 15
         ●───●───●───●───●
         2   6   7  12  15

Prefix Array: [2, 6, 7, 12, 15]

Quick Query: Sum of subarray [1 to 3]?
Answer: prefix[3] - prefix[0] = 12 - 2 = 10 ✅
Verification: arr[1] + arr[2] + arr[3] = 4 + 1 + 5 = 10 ✅
```

Key Characteristics:

- ✓ Works with **negative numbers** (Sliding Window fails!)
- ✓ O(1) subarray sum queries after O(n) preprocessing
- ✓ Often paired with **HashMap** for exact conditions
- ✓ Perfect for "exact" targets: sum = k, sum % k = 0

2. When to Use Prefix Sum 🎯

Decision Flowchart:

```
🎯 WHEN TO USE PREFIX SUM

START: Do you have an array/subarray problem?
  ↓ YES

Question 1: Does it involve NEGATIVES?
  ↓ YES                    ↓ NO
  ✅ Prefix Sum        Consider Sliding Window

Question 2: What are you finding?
  ├─→ Max/Min Sum          → Use Kadane's
  ├─→ Exact Sum = k        → Use Prefix Sum + HashMap
  ├─→ Sum divisible by k   → Use Prefix Sum + HashMap
  ├─→ Equal 0s and 1s      → Use Prefix Sum + HashMap
  └─→ Left sum = Right sum → Use Prefix Sum (simple)

KEY TRIGGERS:
─────────────────────────────────────────────
✅ "Subarray with sum EXACTLY k"
✅ "Count subarrays where sum is divisible by k"
✅ "Find length of subarray with equal 0s and 1s"
✅ "Pivot index where left sum = right sum"
✅ Array contains NEGATIVE numbers
✅ Need to query many subarray sums
```

Recognition Checklist:

Criterion	Description	Example
1. Subarray Problem	Involves contiguous elements	"Find subarray where..."
2. Exact Condition	Sum = k, Sum % k = 0	"Sum equals 5", "divisible by 3"
3. Count or Length	How many? How long?	"Count of subarrays", "maximum length"
4. Negative Numbers	Array has negatives	[1, -3, 5, -2, 4]

3. Prefix Sum vs Kadane vs Sliding Window 🆚

⚠️ Understanding the Differences is CRITICAL!

Each pattern solves different types of problems. Choose wisely!

Comparison Table:

Pattern	Goal	Condition	Negatives?	Data Structure
Kadane's	Find MAX/MIN sum	Optimization (best sum)	✅ Works	Variables (O(1) space)
Sliding Window	Subarray with condition	sum ≥ k, length ≤ k	❌ Fails	Two pointers
Prefix Sum	Count/Length	sum = k, sum % k = 0	✅ Works	HashMap (O(n) space)

Visual Decision Making:

```
❓ WHICH PATTERN TO USE?

Question: "Find maximum subarray sum"
─────────────────────────────────────────────
Goal: OPTIMIZATION (find best)
Condition: COMPARATIVE (is this bigger?)
Answer: ✅ KADANE'S ALGORITHM

Question: "Find length of subarray with sum = k"
─────────────────────────────────────────────
Goal: SPECIFIC VALUE (exactly k)
Condition: EXACT (sum must equal k)
Answer: ✅ PREFIX SUM + HASHMAP

Question: "Find subarray with sum ≥ k" (no negatives)
─────────────────────────────────────────────
Goal: FIND SUBARRAY
Condition: COMPARATIVE (at least k)
Negatives: NO
Answer: ✅ SLIDING WINDOW

Question: "Count subarrays divisible by k"
─────────────────────────────────────────────
Goal: COUNT (how many?)
Condition: EXACT (remainder = 0)
Answer: ✅ PREFIX SUM + HASHMAP
```

💡 Quick Decision Rule:

- Need **MAX/MIN**? → Kadane's
- Exact **sum = k**? → Prefix Sum
- No negatives + **sum ≥ k**? → Sliding Window
- Has negatives + **any exact condition**? → Prefix Sum

4. The Master Template: Red vs Green 🎨

🔑 THE CORE INSIGHT

Every Prefix Sum problem (except pivot index) uses the **"RED PART vs GREEN PART"** strategy!

The Universal Visual Template:

```
🎨 RED vs GREEN: The Master Visualization

You are at index i, calculating CurrentSum from start (0 to i)

┌─────────────────────────────────────────────┐
│            THE ARRAY                          │
│  Index:  0       j          i                 │
│  Array:  [ . . . . . . . . . . . ]            │
│          <────────────────────>               │
│           CurrentSum (Total)                  │
└─────────────────────────────────────────────┘
                     ↓
          ┌──────────┴──────────┐
          ↓                     ↓
┏━━━━━━━━━━━━━┓     ┏━━━━━━━━━━━━━┓
┃  RED PART    ┃     ┃  GREEN PART  ┃
┃ (Remove this)┃     ┃ (Want this!) ┃
┗━━━━━━━━━━━━━┛     ┗━━━━━━━━━━━━━┛
  Sum = ???           Target Condition
  (Query HashMap)     (e.g., Sum = k)

THE LOGIC:
─────────────────────────────────────────────
1. GREEN PART = What we want (target)
2. CURRENT SUM = Red + Green
3. RED PART = CurrentSum - Green
4. Check HashMap: "Have we seen Red Part before?"
5. If YES → Found valid Green Part!

FORMULA:
Red = CurrentSum - Target
```

Step-by-Step Template Process:

Step 1 **Initialize HashMap**

Start with an empty HashMap to store previous states.

`Map<Condition, Frequency or Index>`

Often initialize: `map.put(0, 1)` or `map.put(0, -1)`

Step 2 **Iterate Through Array**

For each index i (end of GREEN PART):

• Calculate CurrentSum (or CurrentDiff, or CurrentRemainder)

• This represents: 0 to i (Red + Green combined)

Step 3 **Define GREEN PART Target**

What do you want GREEN to be?

• Sum = k?

• Sum % k = 0?

• Diff (zeros - ones) = 0?

Step 4 **Calculate RED PART**

What must RED be for GREEN to satisfy the condition?

`Red = CurrentSum - Target`

or

`Red = Current state that would leave target state`

Step 5 **Query HashMap**

Check: `if (map.containsKey(Red))`

• For COUNT problems: `count += map.get(Red)`

• For LENGTH problems: `length = i - map.get(Red)`

Step 6 **Update HashMap**

Store current state for future iterations:

• For COUNT: `map.put(current, frequency + 1)`

• For LENGTH: `if (!map.containsKey(current)) map.put(current, i)`

Example Application:

```
📝 EXAMPLE: Find subarrays with sum = 5

Array: [1, 2, 3, -1, 4]
At index i=3: arr[3] = -1

CurrentSum (0 to 3) = 1 + 2 + 3 + (-1) = 5
                      └─────┬─────┘ └─┬─┘
                          RED?     GREEN?

Question: Can GREEN PART have sum = 5?

If GREEN = 5:
  Red + Green = CurrentSum
  Red + 5 = 5
  Red = 0

Check HashMap: "Have I seen prefix sum = 0 before?"
  → YES! (at index -1, before array started)
  → So subarray [0 to 3] has sum = 5 ✅

Visual:
[ 1   2   3  -1 ]  ← Sum = 5
<─────────────────>
   GREEN PART
(No red part to remove!)
```